



MACHINE LEARNING · COURSE MODULE

Ensemble Learning

Many models, one smarter decision — Bagging, Boosting & Stacking

Bagging

Boosting

Stacking

Why do we need Ensemble Learning?

In machine learning, a single model is often not strong enough on its own. So instead of trusting one model, we combine several models and let them work as a team.



One model may overfit

A single decision tree can memorise the training data and fail on new data.



One model may be too weak

A simple model makes too many mistakes and misses important signal.



One model misses patterns

No single algorithm captures every pattern hidden in the data.



Ensembles are more robust

Combining models gives predictions that are more accurate and stable.



The Wisdom of the Crowd

A group of diverse, independent opinions — combined well — beats almost any single expert.

Single model → limited

Ensemble → accurate & stable

What is Ensemble Learning?

Definition

A technique where multiple models are combined to produce a better final prediction than any single model could achieve alone.

The simple idea

Instead of asking one student to solve a hard problem, you ask many students and combine their answers. The combined answer is usually better.



Better combined answer

Everyday Analogy: Will it rain tomorrow?

Don't ask just one person. Ask several, then combine their opinions:



Weather expert



Farmer



Sailor



Weather app

Combine all opinions → a more reliable forecast.

Three families of Ensemble Learning

Every ensemble method is a different answer to one question: how should we build and combine the individual models?



Bagging

BOOTSTRAP AGGREGATING

Train many models in parallel on random samples, then vote or average.



Reduces variance

Best example: **Random Forest**



Boosting

SEQUENTIAL LEARNING

Train models one after another, each fixing the previous one's mistakes.



Reduces bias

Best example: **XGBoost, AdaBoost**



Stacking

META LEARNING

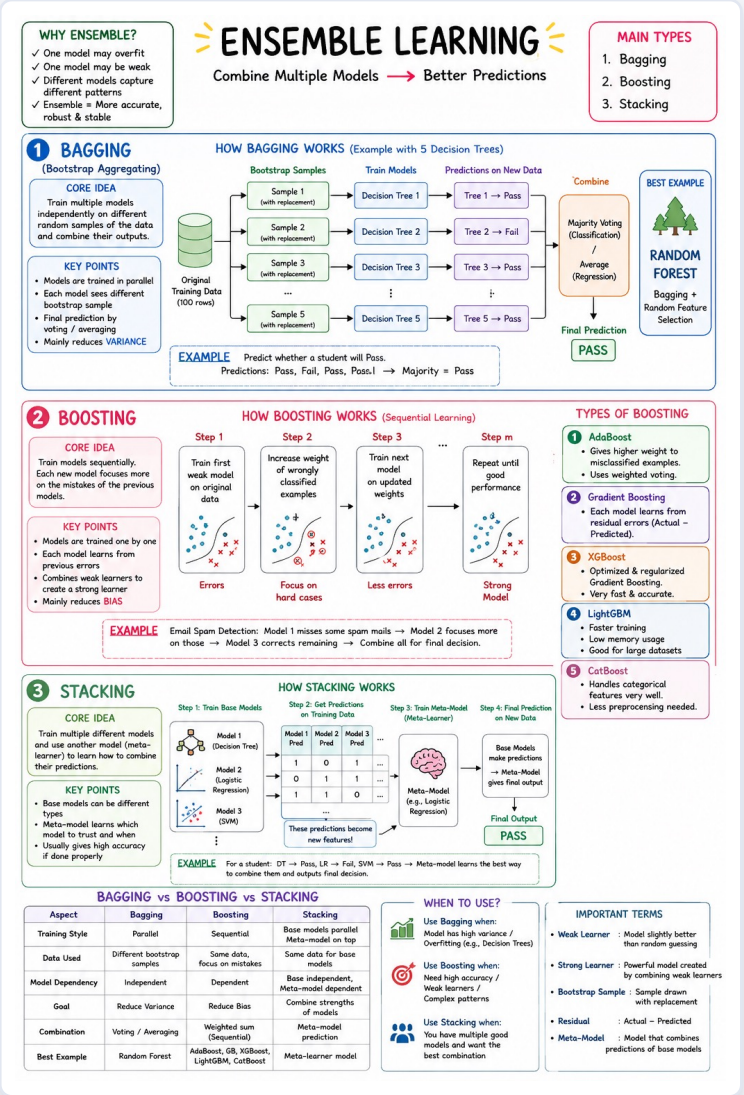
Train different models, then a meta-model learns how to best combine them.



Combines strengths

Best example: **Meta-learner stack**

Ensemble Learning at a glance



How to read this map

● Bagging

Parallel models on bootstrap samples → majority vote / average. Mainly cuts variance.

● Boosting

Sequential weak learners, each focusing on earlier errors. Mainly cuts bias.

● Stacking

Different base models feed a meta-learner that decides how to combine them.

Tip: keep this sheet handy — we unpack each block next.

The base learner: a Decision Tree

Most ensembles are built from many simple models called base learners. The decision tree is the most common one — it splits data with yes/no questions until it reaches a decision.



Asks a question at each node



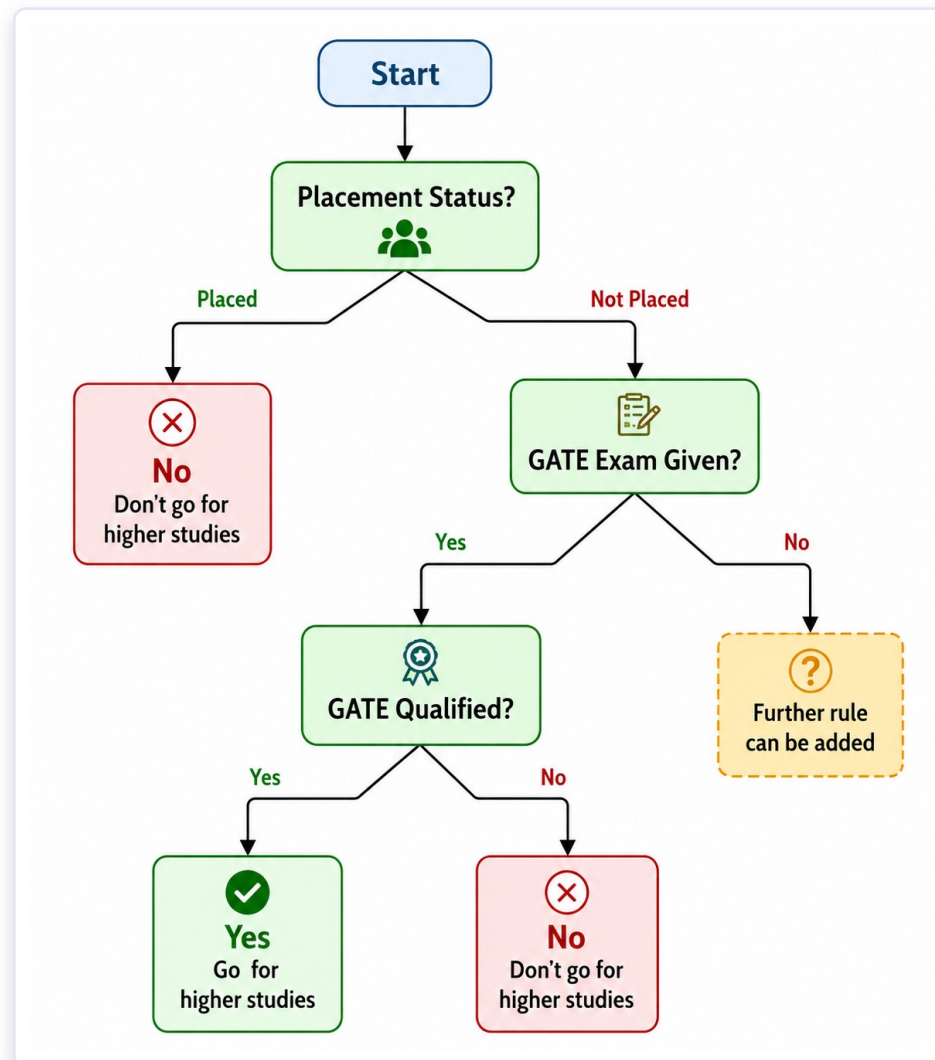
Branches on the answer



Ends in a final decision (leaf)



Why ensembles? A single tree is unstable — a small change in data can change it a lot. Combining many trees fixes this.



Example: should a student go for higher studies?



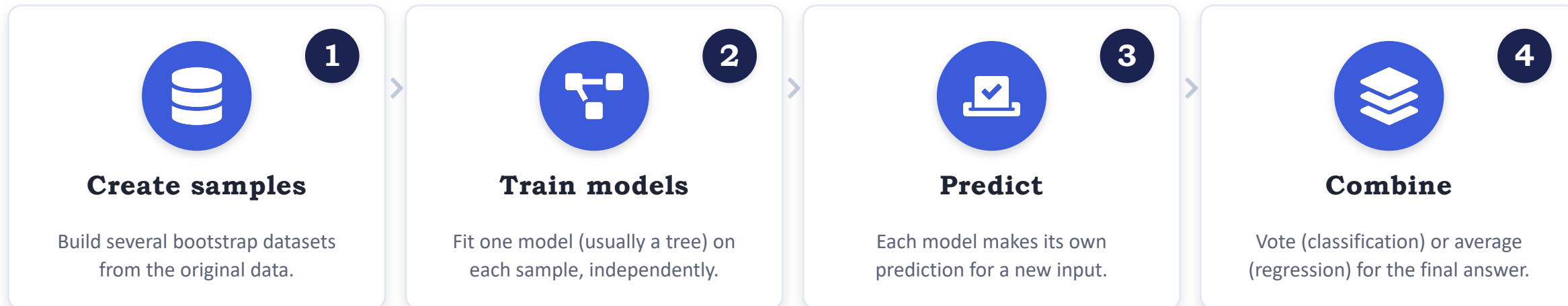
PART 1

Bagging

Bootstrap Aggregating — train many independent models on random samples of the data and combine them by voting or averaging.

What is Bagging?

Bagging = Bootstrap Aggregating. Train many models on different random samples of the data, then combine their outputs.

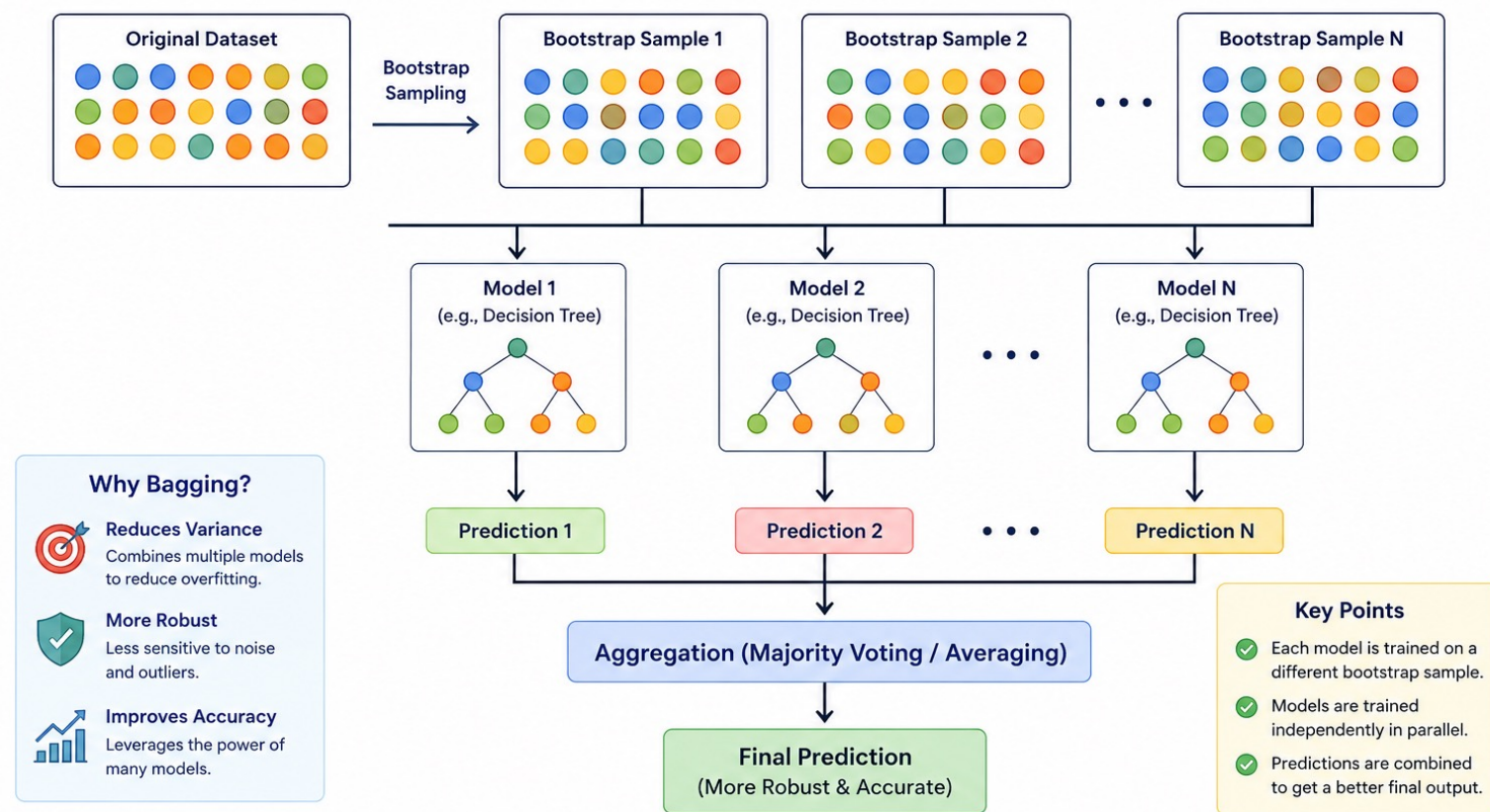


Bootstrap sample = drawn with replacement

The same row can appear several times, and some rows may not appear at all. This randomness makes each model see a slightly different world — which is exactly what creates useful diversity.

How Bagging works, end to end

Ensemble Learning with Bagging

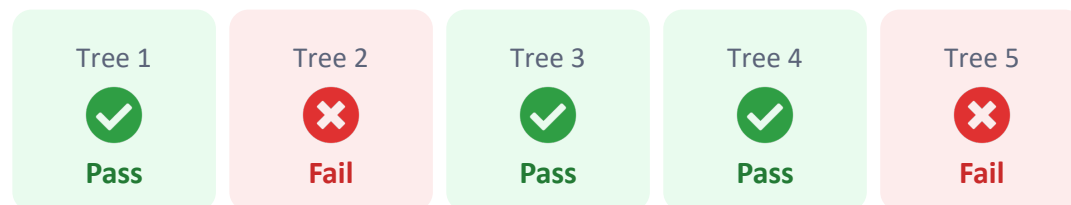


Read it top-down

- 1 Original dataset**
one labelled training set
- 2 Bootstrap samples**
random subsets with replacement
- 3 Train models**
a tree per sample, in parallel
- 4 Aggregate**
majority vote / averaging

A worked example & why it works

Will this student pass? — 5 trees vote



Pass = 3 Fail = 2 → **Final = Pass**

For regression we would average the numbers instead of voting (e.g. 80, 85, 78, 82, 84 → 81.8).



Bagging reduces variance

One tree swings wildly when data changes slightly. Averaging many trees cancels out that noise, giving a stable, robust prediction.

Key characteristics



Trained in parallel

All models are built at the same time.



Independent

No model depends on another's result.



Different samples

Each model sees its own bootstrap data.



Combined output

Voting or averaging gives the final call.

Random Forest — bagging's best example

Random Forest



Random Forest is a modification of bagging that further improves the performance of the model by introducing randomness in the feature selection process.

Key Difference



Bagging

Samples data with replacement.

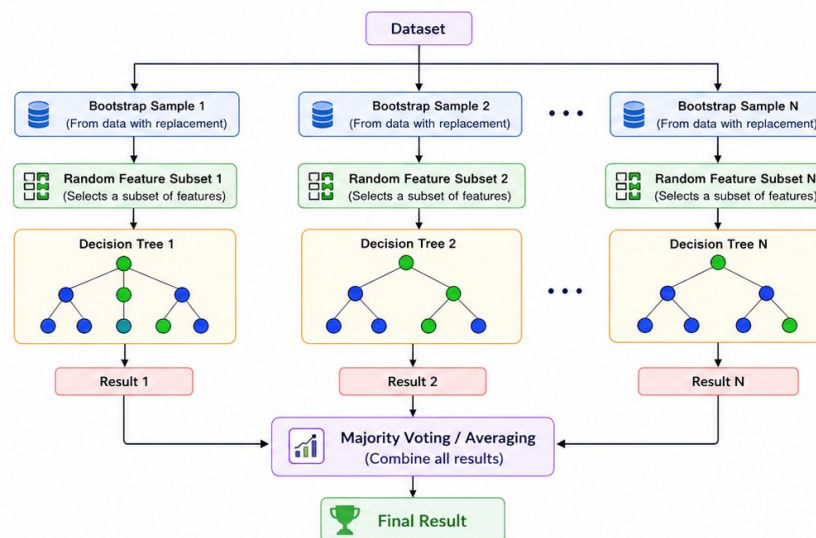


Random Forest

Uses a random subset of features for each tree.

Benefits of Random Forest

- ✓ Reduces overfitting
- ✓ Handles high-dimensional data well
- ✓ More robust and accurate predictions
- ✓ Captures complex relationships



Why it shines



Reduces overfitting



Handles high-dimensional data



Robust, accurate predictions



Captures complex relationships

Extra randomness = less correlated trees

Random Forest = Bagging of decision trees + **random feature selection**



PART 2

Boosting

Sequential learning — weak learners are trained one after another, each new model focusing on the mistakes of the previous ones.

What is Boosting?

Core idea: let the next model focus more on the examples the previous models got wrong. Models learn from mistakes, one after another.



Intuition: a student improving round by round

- 1 Round 1:** Take the test — make some mistakes.
- 2 Round 2:** Focus only on the questions you got wrong.
- 3 Round 3:** Drill the remaining hard questions.

"Learn from mistakes and improve" — that is boosting.

Bagging vs Boosting — the core split



Bagging

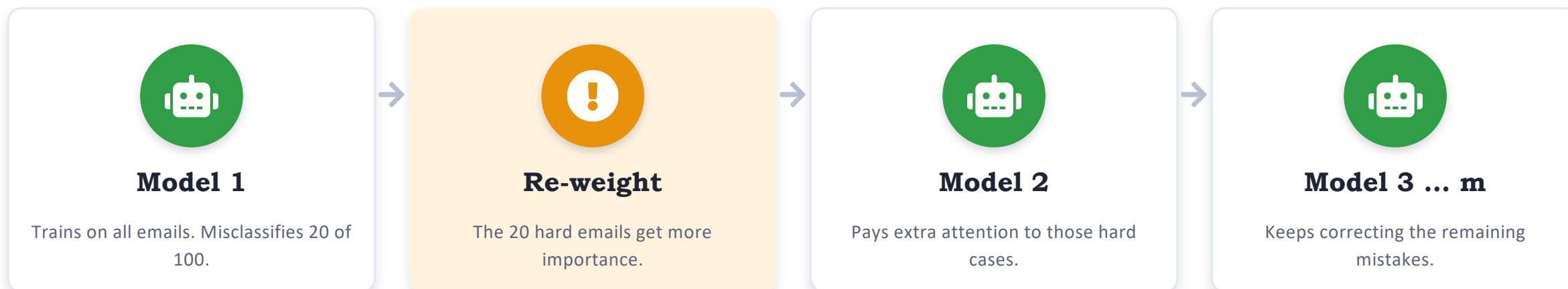
Models work independently, in parallel. Equal say.



Boosting

Models work in sequence, each correcting the previous.

How Boosting works — spam detection



Combine all weak learners → one strong learner with high predictive power.



Boosting reduces bias

It gradually learns difficult patterns a single weak model would miss.



Use it when...

Weak learners need improvement and difficult cases deserve extra attention.

Five boosting algorithms you should know



AdaBoost

ADAPTIVE BOOSTING

Gives higher weight to misclassified points; uses weighted voting.



Gradient Boosting

LEARNS RESIDUALS

Each model predicts the errors (residuals) of the previous one.



XGBoost

EXTREME GB

Optimised, regularised gradient boosting. Fast & accurate.



LightGBM

LIGHT & FAST

Faster training, low memory — great for large datasets.



CatBoost

CATEGORICAL-FRIENDLY

Handles categorical features natively, less preprocessing.

AdaBoost — Adaptive Boosting

Main idea: give higher weight to misclassified points, so the next model pays more attention to them.

- 1 Train a weak learner (often a small tree called a stump).
- 2 Check which examples are wrongly classified.
- 3 Increase the weights of those wrong examples.
- 4 Train the next weak learner on the updated weights.
- 5 Repeat several times.
- 6 Combine all learners with a weighted vote.

Example

Model 1 misclassifies students 2 and 8 → their weights rise.

Model 2 focuses on students 2 and 8.

Model 3 handles the remaining hard students.



Key idea

It focuses on wrongly classified examples — the harder a point is, the more attention future models give it.

Gradient Boosting — learning the residuals

Instead of re-weighting points, each new model predicts the **residual errors** (Actual – Predicted) left by the previous model.

What is a residual?

$$\text{Residual} = \text{Actual value} - \text{Predicted value}$$

Student	Actual	Predicted	Residual
A	90	85	+5
B	70	75	-5
C	80	78	+2

The next model is trained to predict these residuals and correct them.

The correction loop

- 1 Train the first model — predictions aren't perfect.
- 2 Compute residuals (the errors it made).
- 3 Train the next model to predict those residuals.
- 4 Add its correction to the running prediction.
- 5 Recompute residuals and repeat until good.

XGBoost, LightGBM & CatBoost



XGBoost

Extreme Gradient Boosting

- ✓ Regularisation to fight overfitting
- ✓ Parallelised & pruned for speed
- ✓ Handles sparse / missing data
- ✓ King of tabular Kaggle wins



LightGBM

Light Gradient Boosting

- ✓ Very fast training
- ✓ Low memory footprint
- ✓ Scales to large datasets
- ✓ Leaf-wise tree growth



CatBoost

Categorical Boosting

- ✓ Handles categorical features natively
- ✓ Minimal preprocessing needed
- ✓ Strong default performance
- ✓ Great for city / type / branch data



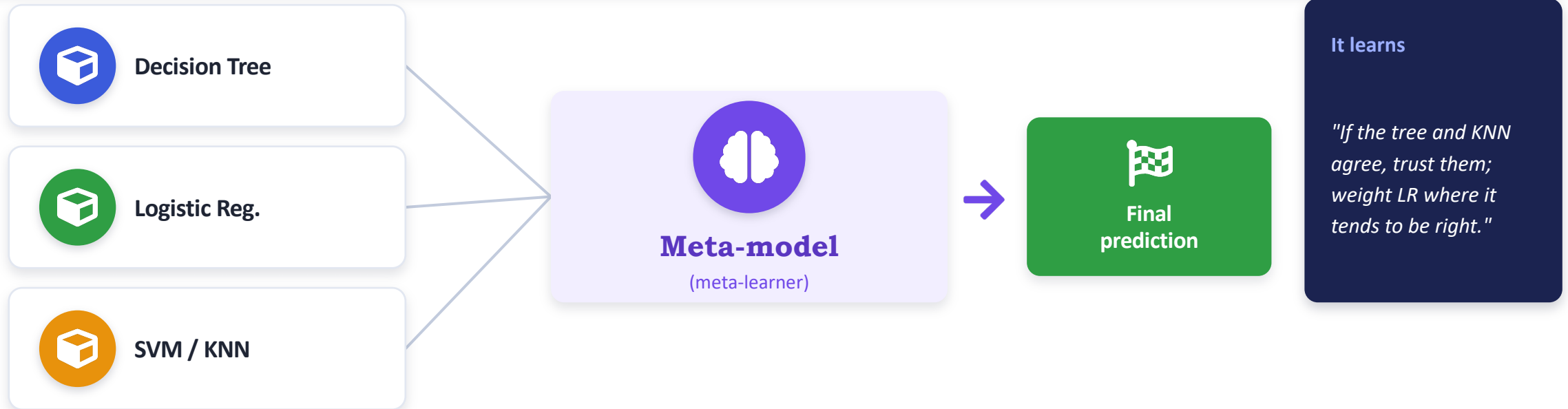
PART 3

Stacking

Meta learning — train several different base models, then let a meta-model learn how to best combine their predictions.

What is Stacking?

Instead of simple voting, **one more model learns which base model to trust, and when.**



Base models can be completely different algorithms — that diversity is the point.

How Stacking works — step by step

- 1


Train base models
 Fit several different models (e.g. Decision Tree, Logistic Regression, KNN).
- 2


Collect predictions
 For each row, record what every base model predicted.
- 3


Predictions become features
 Those predictions form a brand-new training table.
- 4

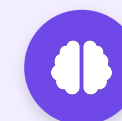

Train the meta-model
 A higher-level model learns from that table how to combine them.
- 5


Final prediction
 The meta-model outputs the final, smarter decision.

Predictions → new features

For one student, the base models said:

DT	LR	KNN
Pass	Fail	Pass



Meta-model decides

These three labels are treated as the inputs the meta-model learns from.

Why Stacking is powerful

Different algorithms capture different kinds of patterns. Stacking keeps each model's strength and lets the meta-learner blend them intelligently.



Decision Tree

Captures simple, rule-based splits.



SVM

Captures margin-based boundaries.



XGBoost

Captures complex nonlinear interactions.

Example — will this customer churn?

Decision Tree



Churn

Random Forest



No Churn

XGBoost



Churn

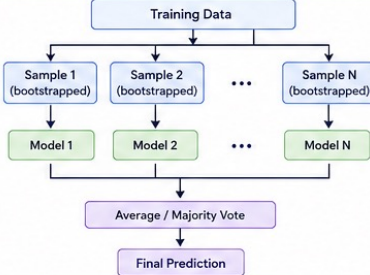
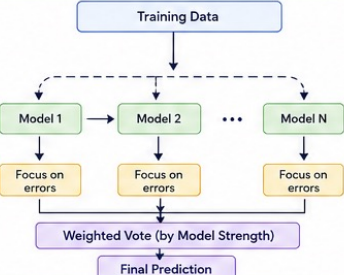
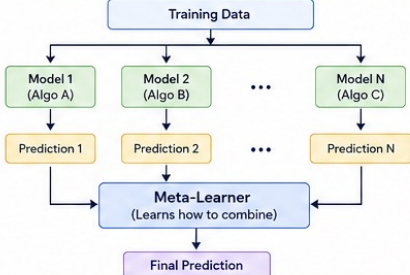


**Meta-model gives the
verdict**

Bagging vs Boosting vs Stacking

Bagging vs Boosting vs Stacking

Three Popular Ensemble Learning Techniques

	BAGGING Bootstrap Aggregating	BOOSTING Sequential Learning	STACKING Meta Learning
 Core Idea	Train many models independently in parallel and combine their predictions.	Train models sequentially where each model focuses on correcting the errors of the previous one.	Train different types of models and use another model (meta-learner) to combine their predictions.
 How It Works			
 Data Used	Different bootstrap samples from the same training data.	Same training data, but models focus more on hard-to-predict instances.	Same training data to train base models; their predictions are used to train the meta-learner.
 Model Training	All models are trained independently and in parallel.	Models are trained sequentially, each depending on the previous one.	Base models are trained (usually in parallel); meta-learner is trained on their outputs.
 Combination of Predictions	Simple average (for regression) or majority vote (for classification).	Weighted average or vote, where stronger models have more influence.	Meta-learner (e.g., Logistic Regression) combines base model predictions.
 Goal	Reduce variance and overfitting.	Reduce bias and underfitting.	Improve generalization by leveraging the strengths of different models.
 Best Used When	Models are high variance (e.g., Decision Trees).	Models are high bias (e.g., Weak Learners like Shallow Trees).	You have diverse models and want the best overall performance.
 Key Takeaway	Multiple models in parallel, independent and equal.	Models in sequence, each trying to correct the previous.	Different models combined by a smarter model.

When should you use each?



Use Bagging when

- ✓ A model is unstable / high variance
- ✓ Overfitting is a concern
- ✓ Trees fluctuate too much
- ✓ You want a simple, robust ensemble

Typical: Random Forest



Use Boosting when

- ✓ You need strong predictive power
- ✓ Weak learners need improving
- ✓ Hard cases need extra attention
- ✓ Data is structured / tabular

Typical: XGBoost, LightGBM



Use Stacking when

- ✓ You have several good, different models
- ✓ You want to combine their strengths
- ✓ You need more than simple voting
- ✓ Top accuracy matters most

Typical: meta-learner stack



PART 4

Advanced Topics

Beyond combining models: bias–variance, voting classifiers, feature importance, hyperparameter tuning and explainability.

The Bias–Variance trade-off

Every model's error is a balance of bias and variance. The goal of ensembles is to reduce total error by managing this balance.



High Bias

Underfitting

Model too simple — misses the real pattern.
High error on train & test.



Good Balance

Just right

Right level of complexity — captures the
pattern, low test error.



High Variance

Overfitting

Model too complex — fits noise. Low train
error, high test error.



Bagging mainly reduces VARIANCE

Averaging many models smooths out the swings of an overfit learner.



Boosting mainly reduces BIAS

Sequentially correcting errors lets simple learners capture hard patterns.

Voting Classifiers — hard vs soft

The simplest way to combine predictions from multiple models.



Hard Voting

Majority vote on predicted class labels.

Model 1

Class A

Model 2

Class B

Model 3

Class A

Votes: A → 2, B → 1

Final prediction: **Class A**



Soft Voting

Average the predicted probabilities per class.

	M1	M2	M3	Avg
Class A	0.70	0.40	0.65	0.583
Class B	0.30	0.60	0.35	0.417

Highest average wins → **Class A**

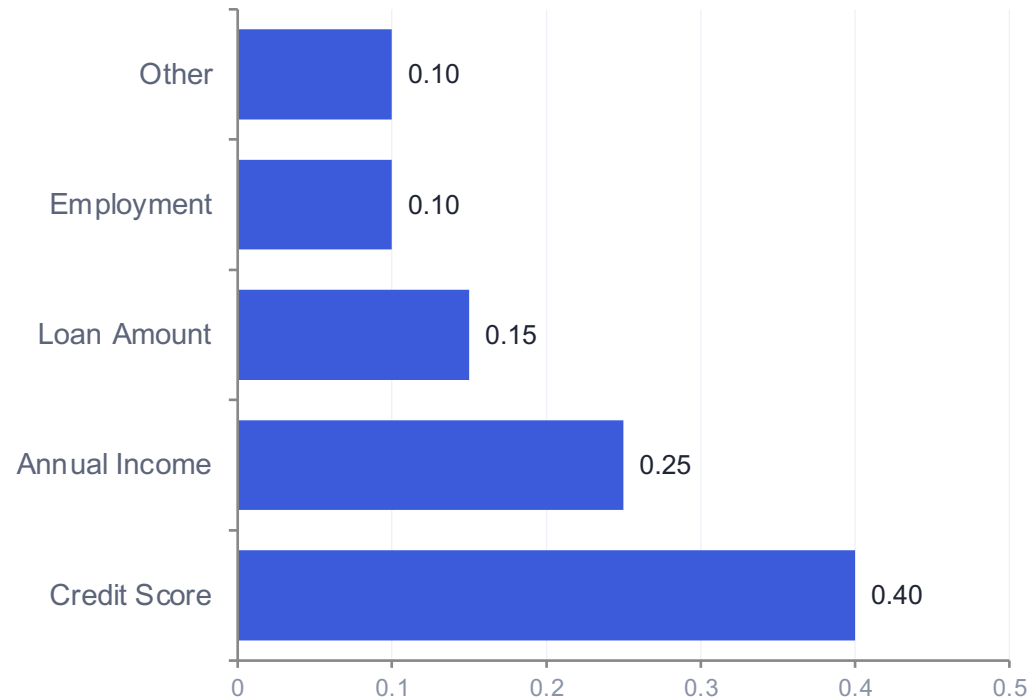
More informative, but needs probability output.

Feature importance & tuning



Feature importance

Which inputs drove the predictions (loan-approval model).



Hyperparameter tuning

The right settings can greatly improve performance.



Grid Search

Try all combinations in a defined grid.



Random Search

Sample random combinations — faster for big spaces.



Bayesian Optimisation

Use past results to search smarter.

Best practice: cross-validate, start small, watch overfitting.

Interpretability & Explainability

Interpretability

Can a human understand the model itself?

Single Decision Tree

Transparency: High

Random Forest

Transparency: Low–Medium

XGBoost / LightGBM

Transparency: Low

Stacked Ensemble

Transparency: Low (complex)

Explainability tools

Why did the model make this prediction?



Feature Importance

Global view of influential features.



SHAP values

How each feature pushed a prediction up or down.



Partial Dependence

Relationship between a feature and the output.



LIME

Local explanation for one prediction.

Why it matters



Builds trust

Know why a decision was made.



Debug models

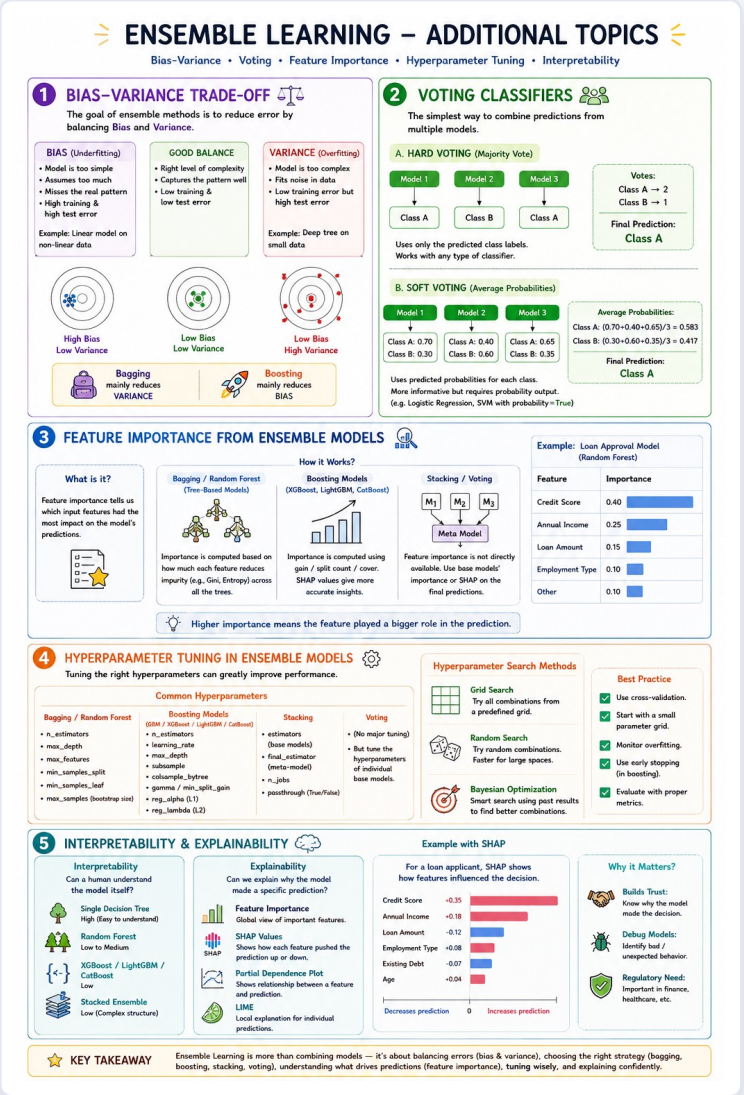
Spot biased or unexpected behaviour.



Regulatory need

Critical in finance & healthcare.

Advanced topics on one page



What's covered here

Bias-Variance

Balance underfitting vs overfitting — the root of model error.

Voting classifiers

Hard (labels) vs soft (probabilities) ways to combine.

Feature importance

Which inputs drive predictions across tree ensembles.

Hyperparameter tuning

Grid / random / Bayesian search to dial in performance.

Interpretability

SHAP, LIME & PDP to explain the why behind outputs.

Key terms & quick viva



Weak learner

A model only slightly better than random guessing (e.g. a small tree).



Strong learner

A powerful model built by combining many weak learners.



Bootstrap sample

A random sample drawn with replacement — used in bagging.



Meta-model

The model in stacking that combines base-model predictions.



Residual

Actual – Predicted; the error gradient boosting learns from.



Quick viva

Bagging → **Random Forest** Misclassified →
AdaBoost Residuals → **Gradient Boosting**

KEY TAKEAWAYS

Ensemble Learning, in one breath



Bagging

Many independent models on bootstrap samples; vote / average. Reduces variance. → Random Forest.



Boosting

Sequential models, each fixing the last's errors. Reduces bias. → AdaBoost, GB, XGBoost.



Stacking

Different base models feed a meta-learner that decides how to combine them.



Ensemble learning is about balancing error (bias & variance), choosing the right strategy, understanding what drives predictions, tuning wisely — and explaining confidently.